Dr. A. Q. Khan Institute of Space and Technology

Visual Programming Lab

Inventory Hub Management System

Name: Taqdees

Semester: CS 6A

Reg No: 232201005

Instructor: Sir Uzair Hassan

April 20, 2026

Contents

1 Introduction **2**

2 System Overview **2**

3 OOP Implementation **2**

3.1 Encapsulation 2

3.2 Inheritance 3

3.3 Polymorphism 3

3.4 Abstraction 4

3.5 Interfaces 4

3.6 Constructors 5

3.7 Enumerations 5

3.8 Delegates and Events 6

3.9 Exception Handling 6

4 MongoDB Integration **7**

5 Docker Implementation **8**

5.1 Running Container 9

6 Admin Module **9**

6.1 Landing Screen & Main Menu 9

6.2 Admin Authentication 10

6.3 Inventory Management 10

6.4 Supplier Management 11

6.5 Order Management 12

6.6 Reports 12

6.7 Database Stats 13

7 Customer Module **14**

7.1 Registration & Login 14

7.2 Customer Dashboard 15

7.3 Search Product 15

7.4 Add to Cart 15

7.5 View Cart 16

7.6 Remove from Cart 16

7.7 Order History 16

7.8 View Profile 17

7.9 Checkout & Invoice 17

8 Conclusion **17**

1 Introduction

Inventory Hub is a console-based inventory and warehouse management system developed in C#. It uses Object-Oriented Programming (OOP) principles to ensure a clean and maintainable structure. MongoDB is used as a NoSQL database for flexible data storage, while Docker enables easy deployment and environment consistency. The system includes separate Admin and Customer panels. The Admin can manage products, suppliers, inventory, and reports, while the Customer can browse products, add items to a cart, and place orders. Overall, the system provides an efficient solution for managing inventory operations in a structured and user-friendly way.

2 System Overview

The application is divided into two primary modules:

- **Admin Module** – Handles product inventory, supplier management, order tracking, generation of various reports, and database statistics. Admins can add, update, delete, and search products, as well as link suppliers to products.
- **Customer Module** – Allows customers to register, log in, browse products, add items to a persistent cart, remove items, view order history, and complete purchases with an invoice. Cart data is saved in the database so it survives logouts.

All data is stored in MongoDB collections, ensuring scalability and flexibility.

3 OOP Implementation

3.1 Encapsulation

Encapsulation hides internal state using properties. In `Product`, `Quantity` and `Price` have validation to ensure data integrity.

```
private int _quantity;
private decimal _price;

public static event StockAlertHandler OnStockAlert;

5 references
protected static void RaiseStockAlert(Product product, string message)
{
    OnStockAlert?.Invoke(product, message);
}

[BsonId]
20 references
public int Id { get; set; }

[BsonElement("name")]
25 references
public string Name { get; set; }

[BsonElement("price")]
10 references
public decimal Price
{
    get => _price;
    set
    {
        if (value < 0) throw new ArgumentException("Price cannot be negative");
        _price = value;
    }
}
```

Figure 1: Encapsulation – Product class

3.2 Inheritance

Specialized product classes (Electronics, Clothing, etc.) inherit from the abstract `Product`, reusing common fields and behaviour.

```
// ==================== DERIVED CLASSES ====================
8 references
public class ElectronicsProduct : Product
{
    [BsonElement("warrantyMonths")]
    4 references
    public int WarrantyMonths { get; set; }
    6 references
    public override ProductType Category => ProductType.Electronics;
    0 references
    public ElectronicsProduct() : base() { }
    4 references
    public ElectronicsProduct(string name, decimal price, int quantity, int threshold, int warrantyMonths)
        : base(name, price, quantity, threshold) => WarrantyMonths = warrantyMonths;
    2 references
    public override string GetDisplayInfo() => $"{base.ToString()} | Warranty: {WarrantyMonths} months";
}
```

Figure 2: Inheritance – derived product classes

3.3 Polymorphism

Derived classes override methods like `GetStockStatus()` and `CheckStockAlert()` to provide type-specific logic.

```
// Base class (Product)
public virtual StockStatus GetStockStatus() { ... }

// Derived class (PerishableProduct)
public override StockStatus GetStockStatus()
{
    if (IsExpired()) return StockStatus.OutOfStock;
    return base.GetStockStatus();
}
```

Figure 3: Polymorphism – overridden methods

3.4 Abstraction

Abstract members (`Category`, `GetDisplayInfo()`) force derived classes to implement essential details, hiding complexity.

```
56 references
public abstract class Product : IIdentifiable, IStockable, IDisplayable
{
    private int _quantity;
    private decimal _price;

    public static event StockAlertHandler OnStockAlert;

    5 references
    protected static void RaiseStockAlert(Product product, string message)
    {
        OnStockAlert?.Invoke(product, message);
    }
```

Figure 4: Abstraction – abstract class and members

3.5 Interfaces

Interfaces `IIdentifiable`, `IStockable`, and `IDisplayable` define contracts that multiple classes implement.

```
1 reference
public interface IStockable
{
    3 references
    void AddStock(int quantity);
    2 references
    bool RemoveStock(int quantity);
    8 references
    StockStatus GetStockStatus();
}

1 reference
public interface IDisplayable
{
    7 references
    string GetDisplayInfo();
}
```

Figure 5: Interface implementations

3.6 Constructors

Parameterised constructors initialise object state. Derived classes call the base constructor before setting their own fields.

```
// ElectronicsProduct
public ElectronicsProduct(string name, decimal price, int quantity, int threshold, int warrantyMonth
s)
    : base(name, price, quantity, threshold) => WarrantyMonths = warrantyMonths;

// Supplier
public Supplier(string name, string contact, string email, string city) : this()
{ Id = IdGenerator.GetNextSupplierId(); Name = name; Contact = contact; Email = email; City = city;
}
```

Figure 6: Constructor usage

3.7 Enumerations

Enums `ProductType`, `StockStatus`, and `OrderStatus` define fixed value sets for categories, stock levels, and order lifecycle.

```
13 references
public enum StockStatus
{
    InStock,
    LowStock,
    OutOfStock
}

19 references
public enum OrderStatus
{
    Pending,
    Processing,
    Shipped,
    Completed,
    Cancelled
}
```

Figure 7: Enumeration definitions

3.8 Delegates and Events

Delegates `StockAlertHandler` and `OrderStatusChangedHandler` define callbacks. The static event `OnStockAlert` notifies the UI of stock changes.

```
// ==================== DELEGATES AND EVENTS ====================
public delegate void StockAlertHandler(Product product, string message);
public delegate void OrderStatusChangedHandler(Order order, OrderStatus oldStatus, OrderStatus newStatus);
```

Figure 8: Delegate and event declarations

```
public static event StockAlertHandler OnStockAlert;
protected static void RaiseStockAlert(Product product, string message)
{
    OnStockAlert?.Invoke(product, message);
}
```

Figure 9: Event subscription and handler

3.9 Exception Handling

`try{catch` blocks in database and UI operations catch errors and display user-friendly messages, preventing crashes.

```
1 reference
static async Task ShowDatabaseStats()
{
    try
    {
        var stats = await DatabaseHelper.GetDatabaseStats();
        ConsoleUI.ShowDatabaseStats(
            dbName: stats.DatabaseName,
            isConnected: stats.ConnectionStatus == "Connected",
            productCount: (int)stats.ProductCount,
            supplierCount: (int)stats.SupplierCount,
            customerCount: (int)stats.CustomerCount,
            orderCount: (int)stats.OrderCount,
            totalRevenue: (decimal)stats.TotalRevenue
        );
    }
    catch (Exception ex)
    {
        ConsoleUI.ShowError($"Error: {ex.Message}");
        ConsoleUI.Pause();
    }
}
```

Figure 10: Exception handling example

4 MongoDB Integration

MongoDB, a NoSQL database, stores all application data. The `DatabaseHelper` class connects to a local instance and performs asynchronous CRUD operations.

The database `InventoryHubDB` contains the following collections:

- **Products** – Stores product details including category-specific attributes, stock levels, and supplier linkage. It supports polymorphic deserialization for different product types.
- **Suppliers** – Maintains supplier information such as name, contact, email, and city.
- **Orders** – Records customer orders, including ordered items, total amount, order date, and status.
- **Customers** – Holds registered customer profiles with login credentials, personal details, and cart data.
- **Carts** – Persists each customer's shopping cart across sessions, using the customer ID as a reference.

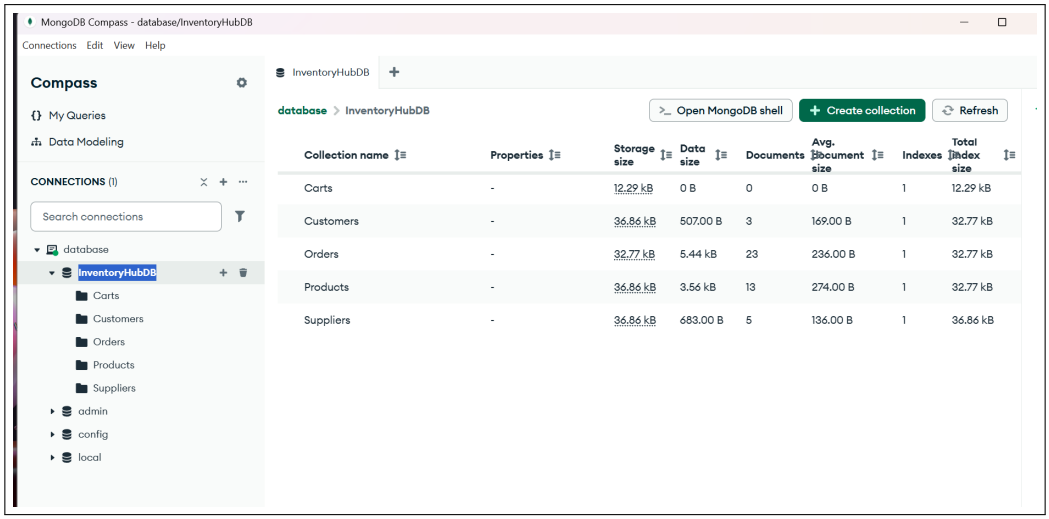

Figure 11: MongoDB database view in Compass

5 Docker Implementation

Docker containerises the MongoDB service, ensuring a consistent environment without manual installation.

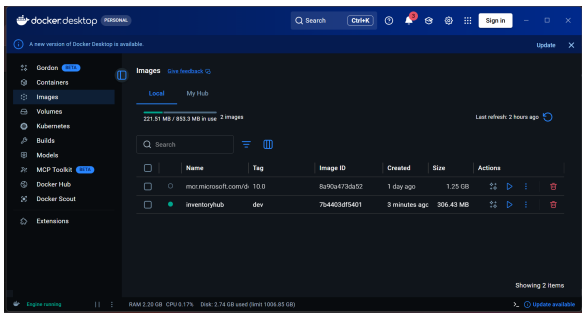

Figure 12: Docker image

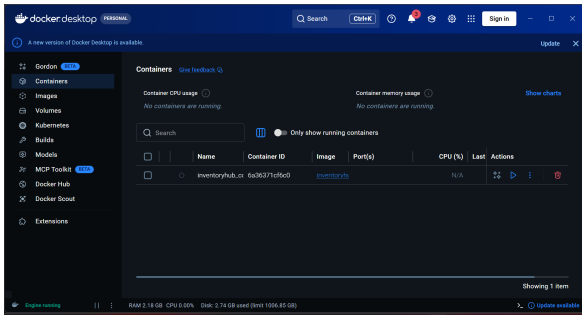

Figure 13: Docker container

5.1 Running Container

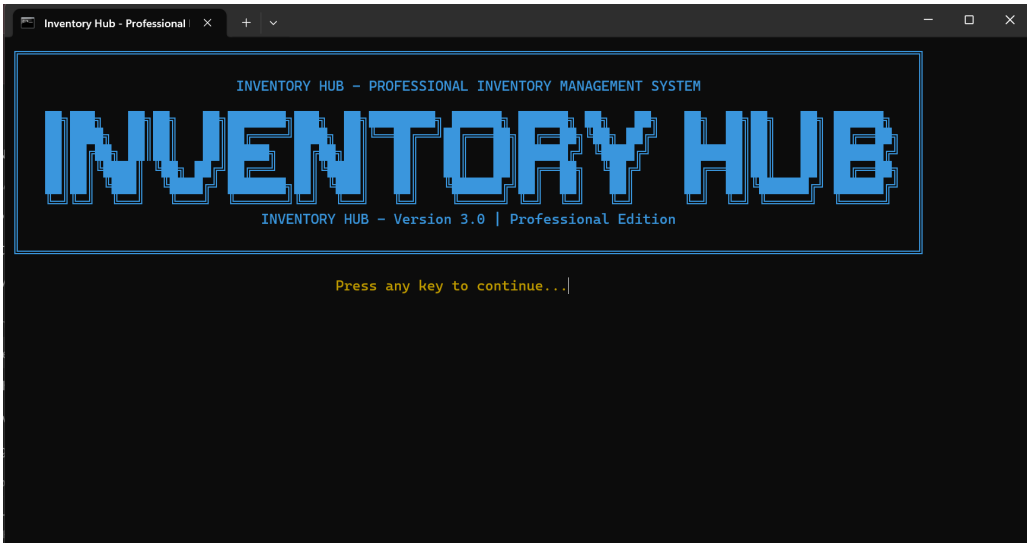

Figure 14: Running container

6 Admin Module

The Admin module provides full control over the system. It allows the administrator to manage products, suppliers, orders, and generate business reports. All operations are protected behind a login screen.

6.1 Landing Screen & Main Menu

Figure 15: Landing Screen

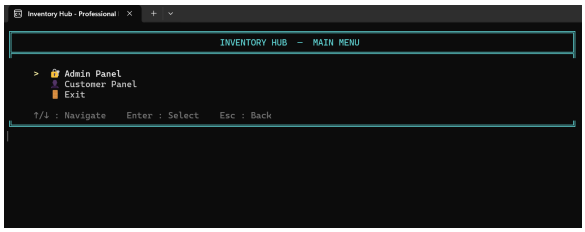

Figure 16: Main Menu

6.2 Admin Authentication

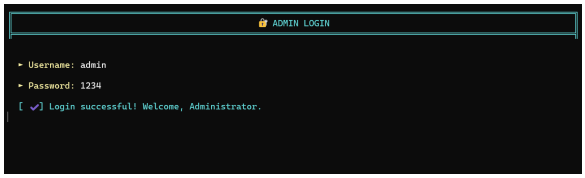

Figure 17: Admin Login

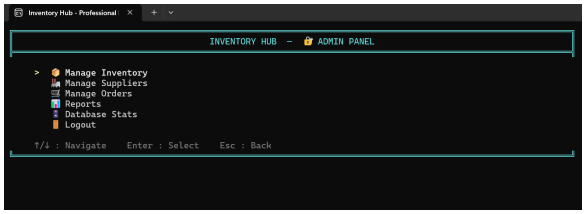

Figure 18: Admin Panel

6.3 Inventory Management

Figure 19: Inventory Menu

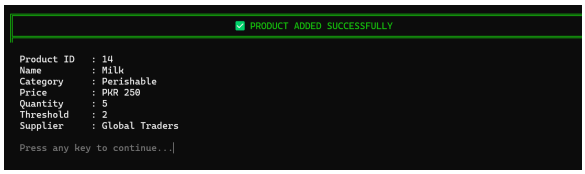

Figure 20: Add Product

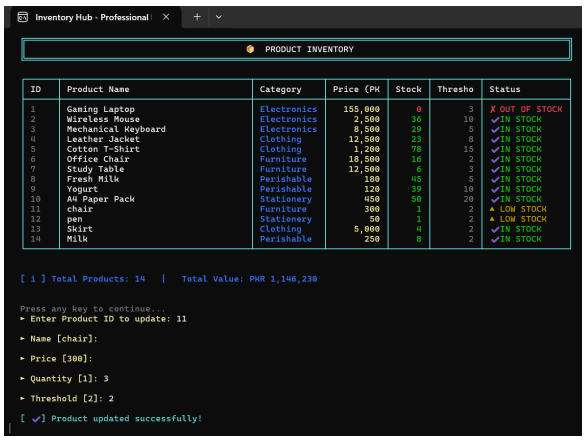

Figure 21: Update Product

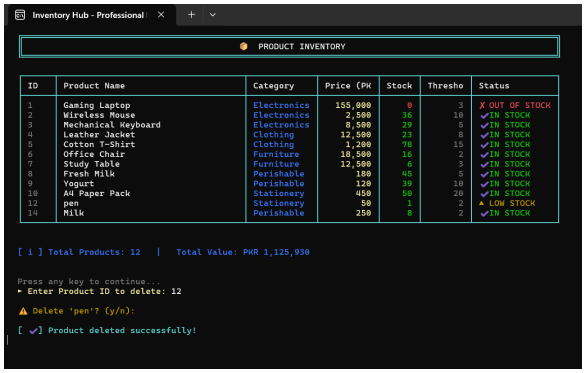

Figure 22: Delete Product

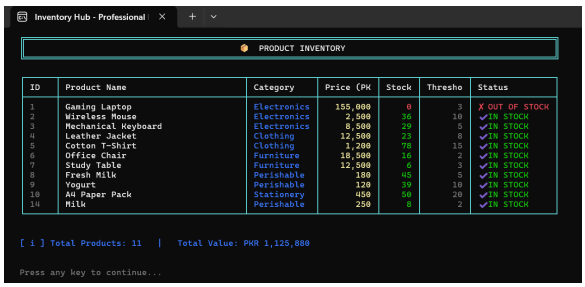

Figure 23: View Products

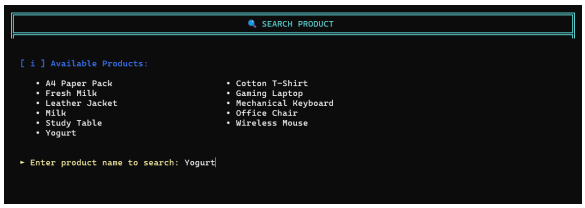

Figure 24: Search Product

6.4 Supplier Management

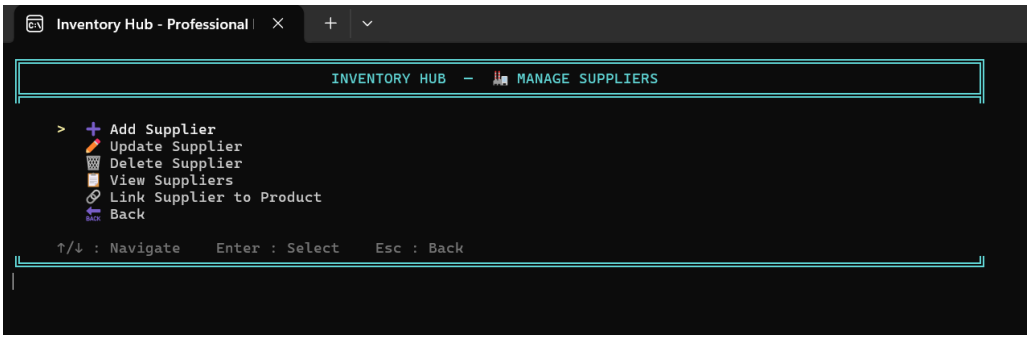

Figure 25: Supplier Menu

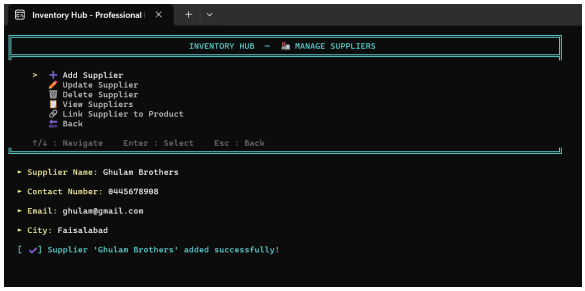

Figure 26: Add Supplier

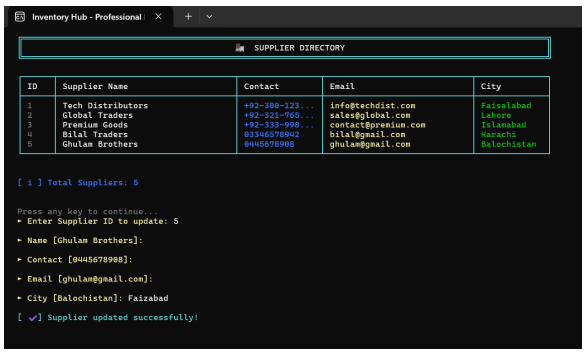

Figure 27: Update Supplier

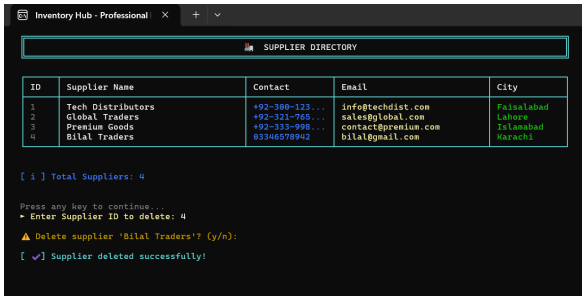

Figure 28: Delete Supplier

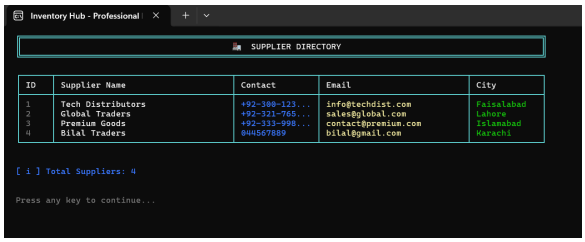

Figure 29: View Suppliers

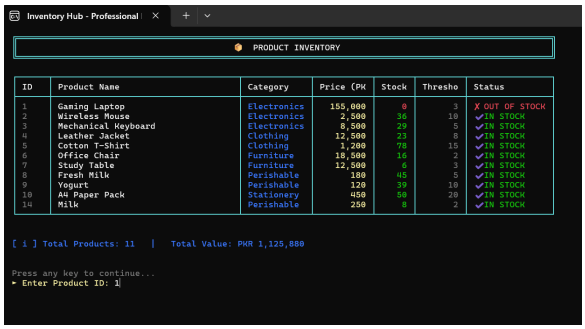

Figure 30: Map Supplier to Product

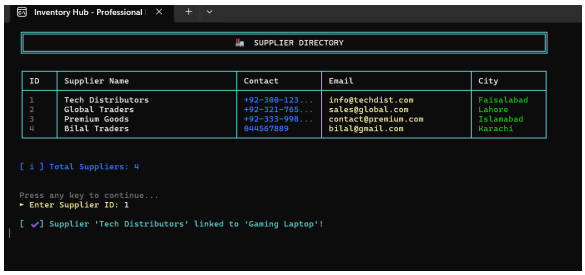

Figure 31: Supplier mapped successfully

6.5 Order Management

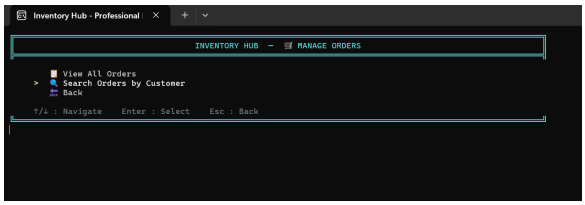

Figure 32: Order Menu

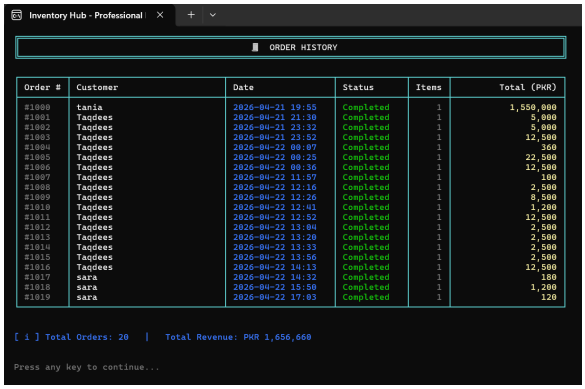

Figure 33: Order History

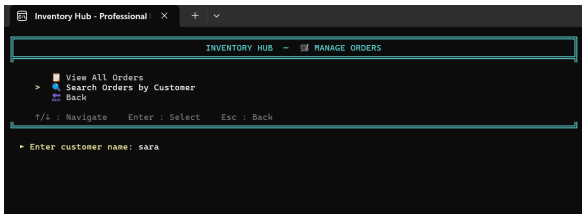

Figure 34: Search Order – input

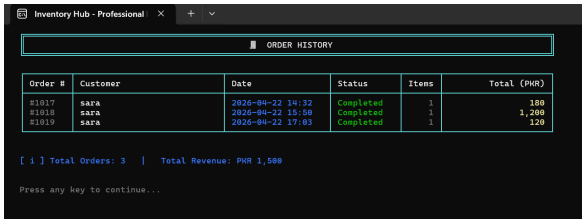

Figure 35: Search Order – result

6.6 Reports

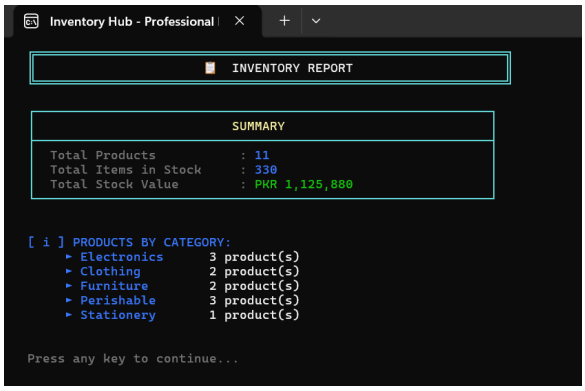

Figure 36: Inventory Report

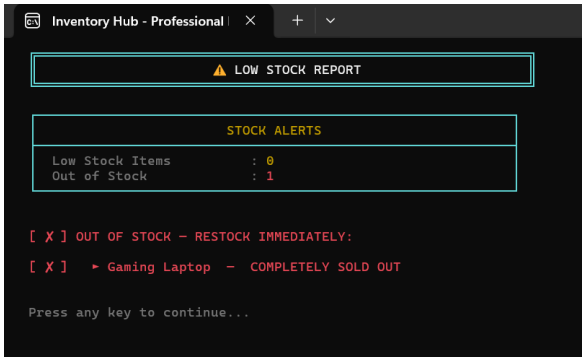

Figure 37: Low Stock Report

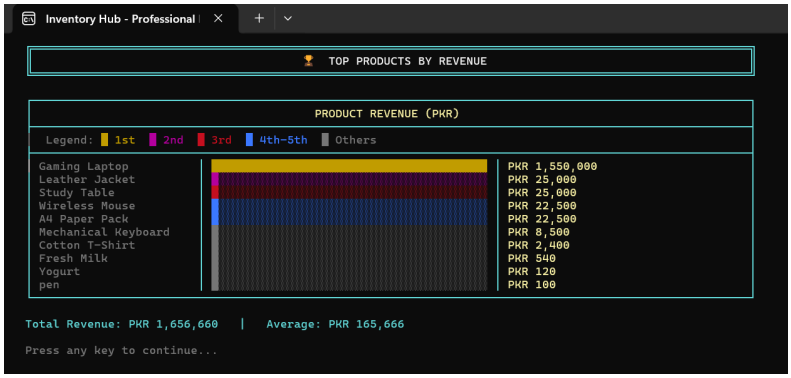

Figure 38: Top Products Revenue

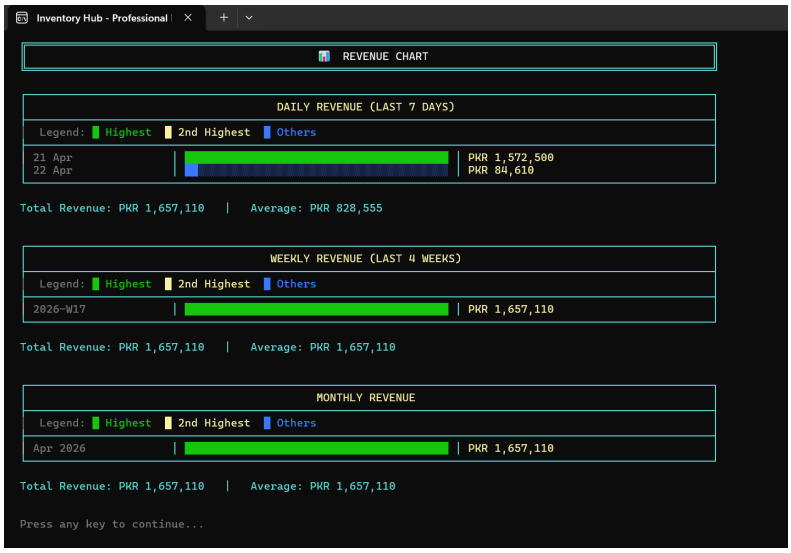

Figure 39: Revenue Chart

6.7 Database Stats

The Database Statistics screen displays an overview of the MongoDB database, including the number of records in each collection and total revenue.

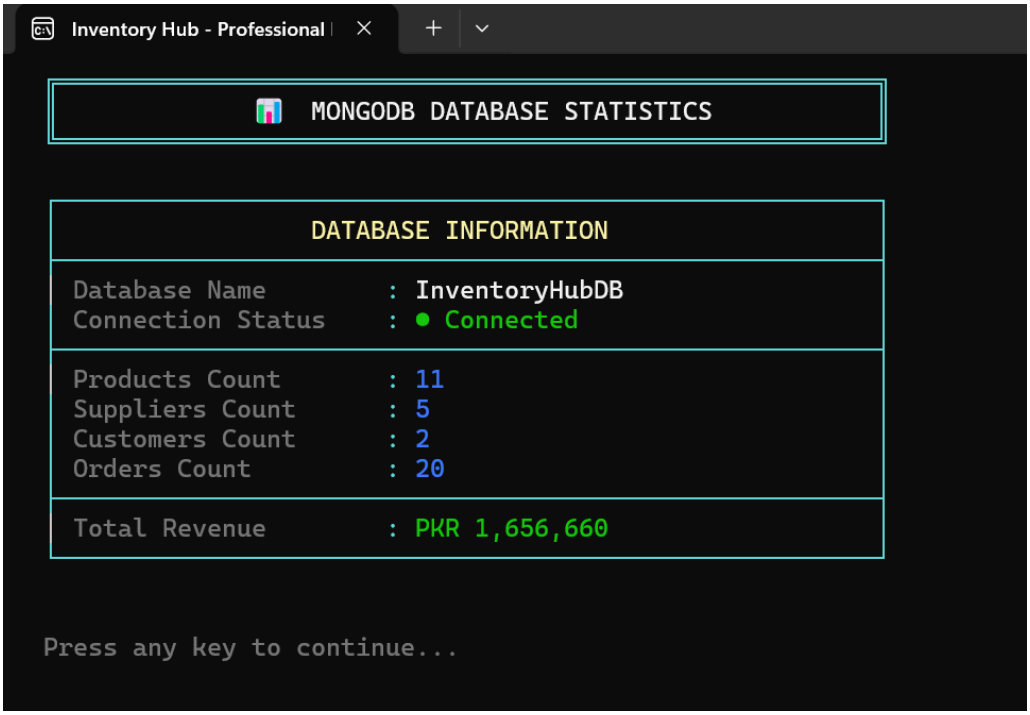

Figure 40: Database Stats

7 Customer Module

The Customer module provides an intuitive shopping experience. It enables users to register, log in, browse products, manage a persistent shopping cart, and place orders with a detailed invoice.

7.1 Registration & Login

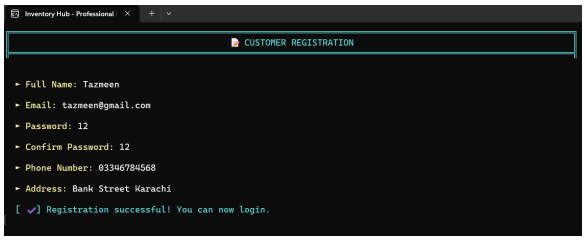

Figure 41: Customer Registration

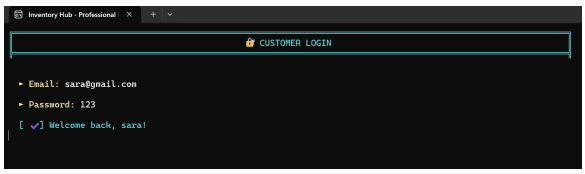

Figure 42: Customer Login

7.2 Customer Dashboard

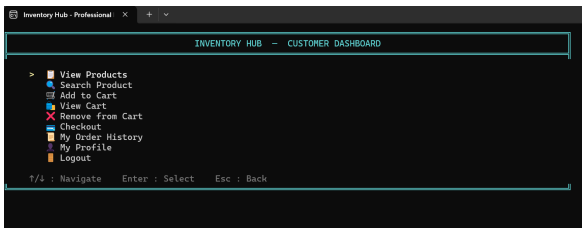

Figure 43: Customer Menu

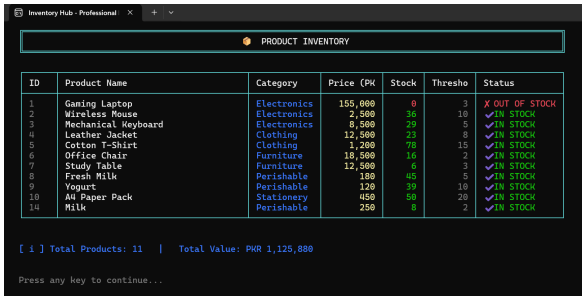

Figure 44: View Products

7.3 Search Product

Figure 45: Search Product

7.4 Add to Cart

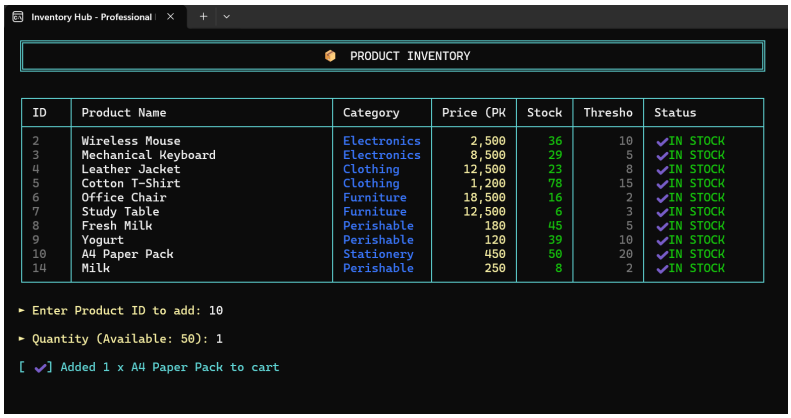

Figure 46: Add to Cart

7.5 View Cart

Figure 47: View Cart

7.6 Remove from Cart

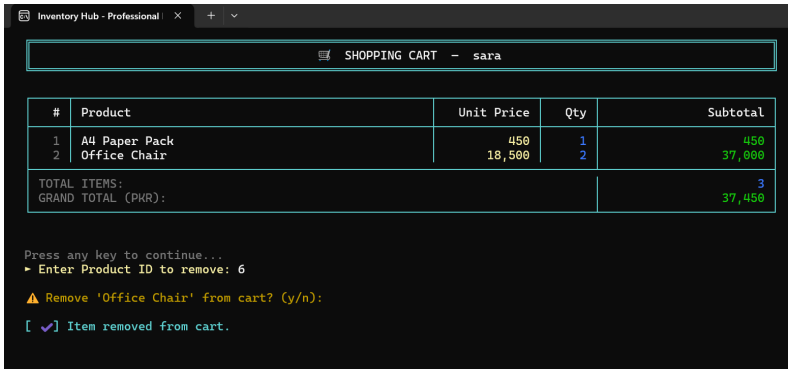

Figure 48: Remove from Cart

7.7 Order History

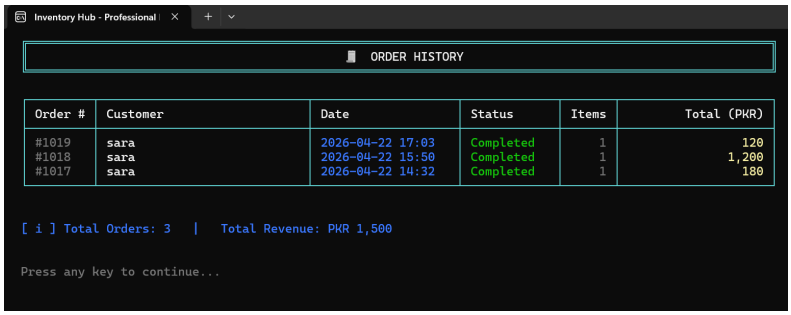

Figure 49: Order History

7.8 View Profile

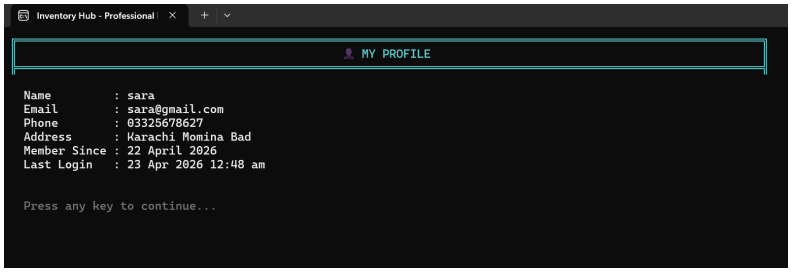

Figure 50: View Profile

7.9 Checkout & Invoice

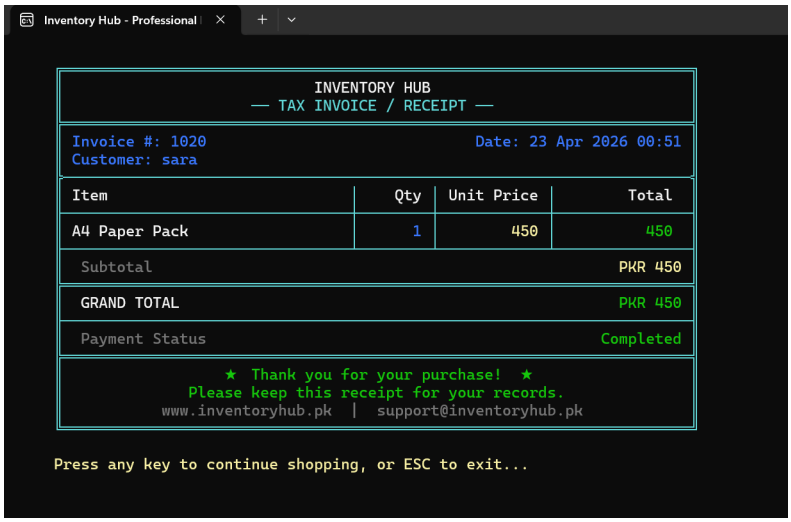

Figure 51: Checkout / Invoice

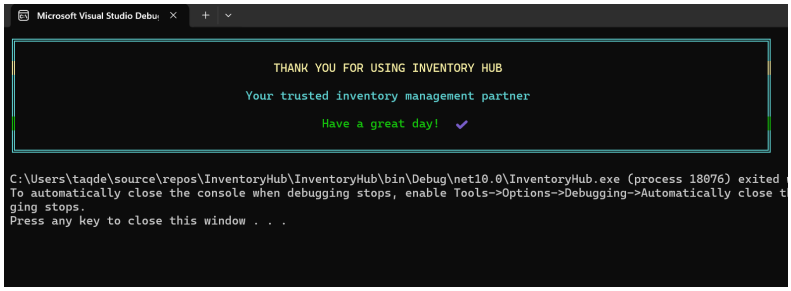

Figure 52: Ending Screen

8 Conclusion

The Inventory Hub project successfully demonstrates OOP principles, MongoDB integration, and Docker containerisation. The console-based UI offers a professional experience for both admin and customers, with persistent carts and detailed reporting.